Factory Planning

Problem

- Find the most profitable production plan (maximize)

Sets

- Products: $p \in Products = \{1(PROD1), 2(PROD2), ..., 7(PROD7)\}$
- Machine types: $m \in Machines = \{1(Grinding), 2(Vertical\ drilling),$ $3(Horizontal\ drilling), 4(Boring), 5(Planing)\}$
- Months: $t \in Months = \{1(Jan), ..., 6(Jun)\}$

Parameters

- Profit for product p: $Profit_p$
- Required process time on machine of type m to produce product p: $ProcessTime_{m,p}$
- Number of machines of type m: $NumMach_m$
- Number of machines of type m that are down for repairs in month t: $MachineRepairs_{m,t}$
- Demand, i.e. max sale of, product p in month t: $Demand_{p,t}$
- Number of working hours pr. month: $WorkingHoursPrMonth = 2 \cdot 8 \cdot 24$ (24 days with two 8 hour shifts each)
- Storage capacity: $StorageCapacity = 100$
- Storage cost pr. month pr. unit: $StorageCost = 0.5$
- Required end stock for each product: $EndStock = 50$

Decision variables

- $x_{p,t}$: amount of product p produced in month t
- $y_{p,t}$: amount of product p sold in month t
- $v_{p,t}$: amount of product p stored in month t

Model

Objective:

- The objective is to maximize profit:

$$\sum_{p}\sum_{t} Profit_p \cdot y_{p,t} - StorageCost \cdot v_{p,t}$$

Constraints:

- Monthly capacity limit on each machine type, taking into account that repairs reduce the available work hours::

$$\sum_{p} ProcessTime_{m,p} \cdot x_{p,t} \leq$$

$$(NumMach_m - MachineRepairs_{m,t}) \cdot WorkingHoursPrMonth \quad \forall m, t$$

- Balance constraint (notice we work with ULTIMO storage):

$$v_{p,t} = v_{p,t-1}|(t > 1) + 0|(t = 1) + x_{p,t} - y_{p,t} \quad \forall p, t$$

- Storage capacity limit:

$$v_{p,t} \leq StorageCapacity \quad \forall p, t$$

- End storage requirement:

$$v_{p,6} = EndStock \quad \forall p$$

- Limit on sale

$$y_{p,t} \leq Demand_{p,t} \quad \forall p, t$$

- Non-negative decisions:

$$x_{p,t}, y_{p,t}, v_{p,t} \geq 0 \quad \forall p, t$$

Notice that the costs depend on how you model storage. In the above we have modeled storage as being what you have in store at the end of a month, referred to as ULTIMO storage. This way, we will not pay storage cost for what was in store before the first month, but we will pay storage cost for the end storage. Oppositely if you were to model PRIMO storage, you would pay for the initial storage but not the final. In the following pages, we show first the Julia program for ULTIMO storage, then PRIMO storage.

The full model in Julia/JuMP with ULTIMO storage constraints , available with the name

`FactoryPlanning_ULTIMO_ANS.jl`

from the book web-site, is given below:

```julia
#************************************************************************
# Factory Planning Assignment ULTIMO-STORAGE,
# "Mathematical Programming Modelling" (42112)

#************************************************************************
# Intro definitions
using JuMP
using HiGHS
#************************************************************************


#************************************************************************
# PARAMETERS
Machines = ["Grinding","Vertical drilling",
            "Horizontal drilling","Boring","Planing"]
M=length(Machines)
Products = ["PROD1","PROD2","PROD3","PROD4",
            "PROD5","PROD6","PROD7"]
P=length(Products)
Time = ["Jan" "Feb" "Mar" "Apr" "May" "Jun"]
T = length(Time)

# data
Profit = [10 6 8 4 11 9 3]
ProcessTime = # machine,product
    [0.5  0.7  0.0  0.0  0.3  0.2  0.5;  # Grinding
     0.1  0.2  0.0  0.3  0.0  0.6  0.0;  # Vertical drilling
     0.2  0.0  0.8  0.0  0.0  0.0  0.6;  # Horizontal drilling
     0.05 0.03 0.0  0.07 0.1  0.0  0.08; # Boring
     0.0  0.0  0.01 0.0  0.05 0.0  0.05  # Planing
     ]
```

```
MachineRepairs = #machine, time
        #Jan #Feb #Mar #Apr #May #Jun
        [1.0  0.0  0.0  0.0  1.0  0.0; #Grinding
        0.0  0.0  0.0  1.0  1.0  0.0;  #V drill
        0.0  2.0  0.0  0.0  0.0  1.0;  #H drill
        0.0  0.0  1.0  0.0  0.0  0.0;  #Boring
        0.0  0.0  0.0  0.0  0.0  1.0]  #Planing

NumMach = [4 2 3 1 1]

StorageCap = 100
StorageCost = 0.5
EndStock = 50
WorkingHoursPrMonth = 24*8*2 # no working days, 2 shifts of 8 hours

Demand = # product, month
    #Jan   #Feb  #Mar  #Apr  #May  #Jun
    [500   600   300   200   0     500;  #PROD1
     1000  500   600   300   100   500;  #PROD2
     300   200   0     400   500   100;  #PROD3
     300   0     0     500   100   300;  #PROD4
     800   400   500   200   1000  1100; #PROD5
     200   300   400   0     300   500;  #PROD6
     100   150   100   100   0     60;   #PROD7
     ]

#*************************************************************************


#*************************************************************************
# Model
factoryplanning = Model(HiGHS.Optimizer)

# production of product p in month t
@variable(factoryplanning, x[1:P,1:T] >= 0)

# sale of product p in month t
@variable(factoryplanning, y[1:P,1:T] >= 0)

# storage of product p in THE END of month t
@variable(factoryplanning, s[1:P,1:T] >= 0)


# maximize profit
@objective(factoryplanning, Max,
```

```
          sum(Profit[p]*y[p,t] for p=1:P, t=1:T) -
          sum(StorageCost*s[p,t] for p=1:P, t=1:T))

# machine capacity limit constraint
@constraint(factoryplanning, machine_cap[m=1:M,t=1:T],
            sum(ProcessTime[m,p]*x[p,t] for p=1:P) <=
            (NumMach[m] - MachineRepairs[m,t])*WorkingHoursPrMonth)

# storage balance constraint
@constraint(factoryplanning, [p=1:P, t=1:T],
            s[p,t] == x[p,t] - y[p,t] + (t>1 ? s[p,t-1] : 0) )

# end storage
@constraint(factoryplanning, [p=1:P],
            s[p,6] == EndStock)

# max storage
@constraint(factoryplanning, [p=1:P, t=1:T],
            s[p,t] <= StorageCap)

# demand limit
@constraint(factoryplanning, [p=1:P, t=1:T],
            y[p,t] <= Demand[p,t])

print(factoryplanning)
#************************************************************************


#************************************************************************
# solve
optimize!(factoryplanning)
println("Termination status: $(termination_status(factoryplanning))")
#************************************************************************


#************************************************************************
# Report results
println("-------------------------------------");
if termination_status(factoryplanning) == MOI.OPTIMAL
    println("RESULTS:\n")
    println("objective = $(objective_value(factoryplanning))")
    println("Positive shadowprices for machine capacity constraints:")
    for m=1:M
        for t=1:T
            if shadow_price(machine_cap[m,t])>0
                println("Machine \"$(Machines[m])\" in $(Time[t]
```

```
                ), : $(shadow_price.(machine_cap[m,t]))")
            end
        end
    end
else
    println("  No solution")
end
println("--------------------------------------");
#**************************************************************************

#**************************************************************************
println("Successfull end of $(PROGRAM_FILE)")
#**************************************************************************
```

Comments to the Julia/JuMP program:

- Notice: The smart way the planned machine maintenance is included by simply reducing the number of available machines: $NumMach_m - MachineRepairs_{m,t}$. This makes the FactoryPlanning2 assignment much easier.
- Notice: The reason that it is ULTIMO storage is because in the constraint: $v_{p,t} = v_{p,t-1} + x_{p,t} - y_{p,t}$ this months storage has the same time index as x and y.

The full model in Julia/JuMP with PRIMO storage constraints , available with the name

`FactoryPlanning_PRIMO_ANS.jl`

from the book web-site, is given below:

```
#**************************************************************************
# Factory Planning Assignment PRIMO-STORAGE,
# "Mathematical Programming Modelling" (42112)

#**************************************************************************
# Intro definitions
using JuMP
using HiGHS
#**************************************************************************


#**************************************************************************
# PARAMETERS
Machines = ["Grinding","Vertical drilling",
```

```
            "Horizontal drilling","Boring","Planing"]
M=length(Machines)
Products = ["PROD1","PROD2","PROD3","PROD4",
            "PROD5","PROD6","PROD7"]
P=length(Products)
Time = ["Jan" "Feb" "Mar" "Apr" "May" "Jun"]
T = length(Time)

# data
Profit = [10 6 8 4 11 9 3]
ProcessTime = # machine,product
    [0.5  0.7  0.0  0.0  0.3  0.2  0.5;  # Grinding
     0.1  0.2  0.0  0.3  0.0  0.6  0.0;  # Vertical drilling
     0.2  0.0  0.8  0.0  0.0  0.0  0.6;  # Horizontal drilling
     0.05 0.03 0.0  0.07 0.1  0.0  0.08; # Boring
     0.0  0.0  0.01 0.0  0.05 0.0  0.05  # Planing
     ]

MachineRepairs = #machine, time
        #Jan #Feb #Mar #Apr #May #Jun
        [1.0  0.0  0.0  0.0  1.0  0.0; #Grinding
        0.0  0.0  0.0  1.0  1.0  0.0;  #V drill
        0.0  2.0  0.0  0.0  0.0  1.0;  #H drill
        0.0  0.0  1.0  0.0  0.0  0.0;  #Boring
        0.0  0.0  0.0  0.0  0.0  1.0]  #Planing

NumMach = [4 2 3 1 1]

StorageCap = 100
StorageCost = 0.5
EndStock = 50
WorkingHoursPrMonth = 24*8*2 # no working days, 2 shifts of 8 hours

Demand = # product, month
    #Jan   #Feb  #Mar  #Apr  #May  #Jun
    [500   600   300   200   0     500;  #PROD1
     1000  500   600   300   100   500;  #PROD2
     300   200   0     400   500   100;  #PROD3
     300   0     0     500   100   300;  #PROD4
     800   400   500   200   1000  1100; #PROD5
     200   300   400   0     300   500;  #PROD6
     100   150   100   100   0     60;   #PROD7
     ]

#*************************************************************************
```

```
#*************************************************************************
# Model
factoryplanning = Model(HiGHS.Optimizer)

# production of product p in month t
@variable(factoryplanning, x[1:P,1:T] >= 0)

# sale of product p in month t
@variable(factoryplanning, y[1:P,1:T] >= 0)

# storage of product p in THE END of month t
@variable(factoryplanning, s[1:P,1:T] >= 0)


# maximize profit
@objective(factoryplanning, Max,
           sum(Profit[p]*y[p,t] for p=1:P, t=1:T) -
           sum(StorageCost*s[p,t] for p=1:P, t=1:T))

# machine capacity limit constraint
@constraint(factoryplanning, machine_cap[m=1:M,t=1:T],
            sum(ProcessTime[m,p]*x[p,t] for p=1:P) <=
            (NumMach[m] - MachineRepairs[m,t])*WorkingHoursPrMonth)

# storage balance constraint (primo storage version)
@constraint(factoryplanning, balance_con[p=1:P, t=1:T],
            s[p,t] == (t>1 ? x[p,t-1] - y[p,t-1] + s[p,t-1] : 0) )

# end storage (primo storage version)
@constraint(factoryplanning, end_storage_con[p=1:P],
            s[p,6] + x[p,6] - y[p,6] == EndStock)
# max storage
@constraint(factoryplanning, [p=1:P, t=1:T],
            s[p,t] <= StorageCap)

# demand limit
@constraint(factoryplanning, [p=1:P, t=1:T],
            y[p,t] <= Demand[p,t])

print(factoryplanning)
#*************************************************************************


#*************************************************************************
# solve
```

```
optimize!(factoryplanning)
println("Termination status: $(termination_status(factoryplanning))")
#************************************************************************


#************************************************************************
# Report results
println("------------------------------------");
if termination_status(factoryplanning) == MOI.OPTIMAL
    println("RESULTS:\n")
    println("objective = $(objective_value(factoryplanning))")
    println("Positive shadowprices for machine capacity constraints:")
    for m=1:M
        for t=1:T
            if shadow_price(machine_cap[m,t])>0
                println("Machine \"$(Machines[m])\" in $(Time[t]
                ), : $(shadow_price.(machine_cap[m,t]))")
            end
        end
    end
else
    println("  No solution")
end
println("-------------------------------------");
#************************************************************************

#************************************************************************
println("Successfull end of $(PROGRAM_FILE)")
#************************************************************************
```

To address whether additional machines will have an impact you can either try to change the values in $NumMach$ one at a time and rerun the code, or inspect the shadow prices (dual values) of the constraints that limit the available machine capacity. These reflect what would be the change of objective value if the RHS is changed by one unit, note that one unit here is one hour of available machine time.
The conclusion should be that you can increase the profit by acquiring one extra machine of any type except vertical drills (machine type 2) and the biggest profit increase is for acquiring an additional planing machine (machine type 5).